

Unidad 8. Almacenamiento y consulta de datos XML: XQuery

Índice de contenidos

1 Almacenamiento de datos XML.....	2
2 Introducción a XQuery.....	3
3 Trabajar con XQuery.....	3
4 Consultas XQuery: expresiones FLWOR.....	5
4.1 La cláusula “for”.....	6
4.2 La cláusula “let”.....	7
4.3 Las cláusulas “where” y “order by”.....	7
4.4 La cláusula “return”.....	8
4.5 Archivos XQuery.....	9
5 XQuery: operadores y funciones.....	10
6 Referencias.....	11

1 Almacenamiento de datos XML

Muchas aplicaciones requieren el almacenamiento de datos XML. Existen diferentes alternativas. Una es la utilización de bases de datos relacionales (BDR) o bases de datos orientadas u objetos (BDOO). Esto requiere que se realicen las transformaciones necesarias para convertir los XML al modelo de la BD. Por ejemplo, en la actualidad muchos sistemas gestores de BDR tienen soporte para XML, pero aún siguen almacenando toda la información de manera relacional, es decir en forma tabular (tablas, registros y columnas) o caso contrario almacenan todo el documento en formato Binary Large Object (BLOB). La principal característica que brindan estas bases de datos es la capacidad de obtener los resultados de las consultas en formato XML; es por ello que dichas bases de datos pertenecen a la categoría de "XML-enabled database".

Otra alternativa para almacenar datos XML que se está desarrollando en la actualidad son las bases de datos XML nativas, o "native XML database". Las bases de datos nativas definen un modelo lógico para el documento XML (a diferencia de dicho modelo), además, almacenan y recuperan documentos de la misma manera que los XML. Por lo menos este modelo debe incluir atributos como PCDATA, y además preservar el orden de los documentos.

Todas las bases de datos relacionales son centradas en los Datos, pues lo que ellos almacenan en sus campos son datos simples más conocidos como datos atómicos. Una base de datos nativa en XML, no posee campos, ni almacena datos atómicos, lo que ella almacena son documentos XML, por lo tanto a este tipo de bases de datos se les denomina bases de datos centradas en documentos.

Las BD nativas se caracterizan principalmente por:

- Emplear como unidad lógica de almacenamiento el documento XML
- Preservar el orden del documento, las instrucciones de procesamiento, los comentarios, las secciones CDATA y las entidades
- Soportar lenguajes de consulta XML
- No tener ningún modelo de almacenamiento físico subyacente concreto. Pueden ser construidas sobre bases de datos relacionales, jerárquicas, orientadas a objetos o bien mediante formatos de almacenamiento propietarios.

Existen multitud de sistemas de bases de datos nativas, por ejemplo:

- Sistema de almacenamiento propietario: Go XML DB, Extc, XDBM (esta es Open Suorce), etc.
- Sistema de almacenamiento relacional: DBDOM, eXist, XDB (todas Open Source)
- Sistema de almacenamiento orientado a objetos: Birdstep RDM XML, MindSuite XDB, Ozone (esta es Open Source).

Algunas de las características generales de las BD nativas son:

- Almacenamiento de documentos en colecciones: los documentos se suelen agrupar, en función de la información que contienen, en colecciones que a su vez pueden contener otras colecciones. Las colecciones juegan en las bases de datos nativas el papel de las tablas en las DB relacionales.
- Validación de los documentos
- Consultas: la mayoría de las BD XML soportan uno o más lenguajes de consulta. Uno de los más populares es XQuery.
- Indexación XML: se ha de permitir la creación de índices que aceleren las consultas realizadas frecuentemente.

- Creación de identificadores únicos: a cada documento XML se le asocia un identificador único por el que será reconocido dentro del repositorio
- Actualizaciones y borrados: las BD nativas tienen una gran variedad de estrategias para actualizar y borrar documentos. Algunos sistemas soportan el lenguaje XUpdate para llevar a cabo esta funcionalidad.
- Aún no existe un lenguaje estándar que permita la actualización, inserción o eliminación de elementos de un documento XML. Existe un lenguaje que permite realizar actualizaciones en un documento XML pero aún no es un estándar y muchos gestores de este tipo de bases de datos no lo soportan, este lenguaje es Xupdate.

2 Introducción a XQuery

XQuery es un lenguaje diseñado para escribir consultas sobre colecciones de datos expresadas en XML, es decir, para extraer determinados elementos y atributos de documentos XML. Se podría decir que XQuery es para XML lo mismo que SQL para bases de datos relacionales. Una consulta en XQuery es una expresión que lee una secuencia de datos en XML y devuelve como resultado otra secuencia de datos en XML.

Actualmente XQuery es una recomendación del W3C [2], y es soportado por los mayores sistemas de bases de datos (IBM, Oracle, Microsoft, etc.). XQuery es compatible con otros estándares del W3C, tales como XML, Namespaces, XSLT, XPath, y XML Schema.

XQuery ha sido construido sobre la base de XPath y se fundamenta en él para realizar la selección de información y la iteración a través del conjunto de datos. Ambos lenguajes, XQuery y XPath están muy relacionados. XQuery 1.0 comparte el mismo modelo de datos y soporta las mismas funciones y operadores que XPath 2.0.

Entre otras cosas XQuery puede utilizarse para:

- Extraer información para ser usada en un servicio Web
- Generar informes
- Transformar datos XML a XHTML
- Buscar información en documentos Web

3 Trabajar con XQuery

Vamos a ver algunos aspectos básicos del uso de XQuery. Para los ejemplos de este tema vamos a utilizar el documento "libros.xml", cuyo contenido es el siguiente:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<libreria>
<libro categoria="COOKING">
  <titulo leng="en">Everyday Italian</titulo>
  <autor>Giada De Laurentiis</autor>
  <anyo>2005</anyo>
  <precio>30.00</precio>
</libro>
<libro categoria="CHILDREN">
  <titulo leng="en">Harry Potter</titulo>
  <autor>J K. Rowling</autor>
  <anyo>2005</anyo>
  <precio>29.99</precio>
</libro>
```

```

<libro categoria="WEB">
  <titulo leng="en">XQuery Kick Start</titulo>
  <autor>James McGovern</autor>
  <autor>Per Bothner</autor>
  <autor>Kurt Cagle</autor>
  <autor>James Linn</autor>
  <autor>Vaidyanathan Nagarajan</autor>
  <anyo>2003</anyo>
  <precio>49.99</precio>
</libro>
<libro categoria="WEB">
  <titulo leng="en">Learning XML</titulo>
  <autor>Erik T. Ray</autor>
  <anyo>2003</anyo>
  <precio>39.95</precio>
</libro>
</libreria>

```

- XQuery usa funciones para extraer los datos de los documentos XML. La función `doc()` se emplea para poder abrir el fichero y devuelve el documento completo. Por ejemplo, `doc("libros.xml")` devuelve el documento "libros.xml".
- XQuery usa expresiones "path" (rutas) para navegar a través de los diferentes nodos de un documento XML. Por ejemplo, la siguiente expresión se emplea para seleccionar todos los títulos (elementos título) del fichero "libros.xml": `doc("libros.xml")/libreria/libro/titulo`
- XQuery usa predicados para limitar (filtrar) el número de datos que se extraen de un documento XML. El siguiente predicado permite seleccionar todos los elementos "libro" bajo el elemento "librería" que tienen un "precio" inferior a 30: `doc("libros.xml")/libreria/libro[precio<30]`
- Al trabajar con XQuery hay que tener en cuenta las siguientes reglas básicas:
 - o XQuery es sensible a mayúsculas y minúsculas.
 - o Cualquier elemento, atributo o variable de XQuery tiene que ser un identificador válido en XML.
 - o Un valor string o cadena de XQuery puede estar contenido tanto en comillas simples como dobles.
 - o Una variable XQuery se define con un símbolo \$ seguido por un nombre. Por ejemplo, \$libreria.
 - o Un comentario XQuery se inserta dentro de un limitador de inicio (: y un delimitador de fin :). Por ejemplo, (: Hola :)
- XQuery y Namespaces: si el documento XML sobre el que va a realizar una consulta utiliza espacios de nombres, las consultas XQuery deben también utilizarlos. Para ello, es necesario declarar en la consulta los espacios de nombres y sus prefijos, mediante la instrucción:

```

declare namespace prefijo="espacio-de-nombres";
Por ejemplo:

```

```

declare namespace alu="http://www.iespruebas.es/alumnos";
declara el espacio de nombres "http://www.iespruebas.es/alumnos" y le asigna el prefijo "alu". El nombre del espacio de nombres tiene que coincidir con el del documento XML, pero el prefijo puede ser diferente.

```

- Los comentarios en XQuery, a diferencia de XML, van encerrados entre caras sonrientes, tal y como se muestra a continuación.

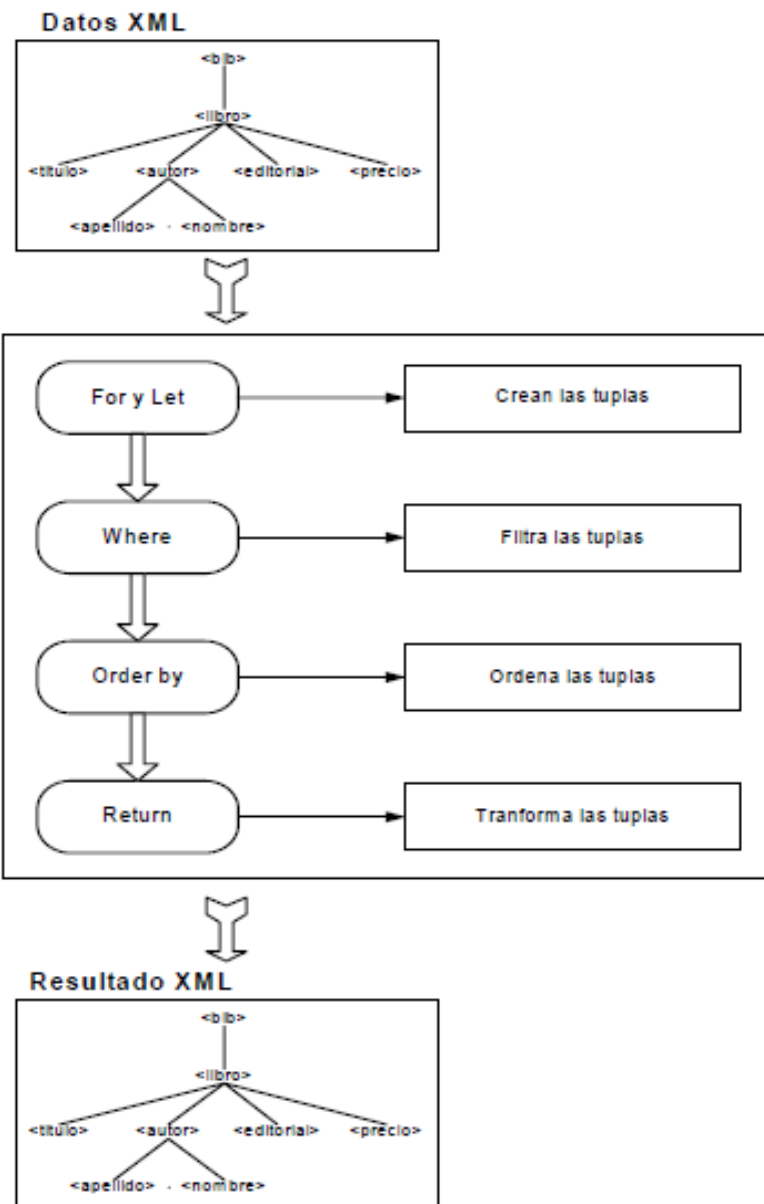
```
(: Esto es un comentario XQuery :)
```

4 Consultas XQuery: expresiones FLWOR

FLWOR (For Let Where Order Return) es una de las expresiones más potentes y típicas de XQuery. Se basa en ligar valores a variables con las cláusulas for y let y emplear esas variables para crear nuevos resultados.

Una expresión FLWOR:

- Comienza con una o más cláusulas for o let en cualquier orden (al menos una de ellas)
- Seguidas por una cláusula where opcional
- Puede aparecer una cláusula order by opcional
- Finaliza con una cláusula return obligatoria



Orden de ejecución y resultado de las cinco cláusulas posibles (fuente [11])

A continuación se describe la función de cada bloque:

- **for**: vincula una o más variables a expresiones escritas en XPath, creando un flujo de tuplas en el que cada tupla está vinculada a una de las variables.
- **let**: vincula una variable al resultado completo de una expresión, añadiendo esos vínculos a las tuplas generadas por una cláusula for o, si no existe ninguna cláusula for, creando una única tupla que contenga esos vínculos.
- **where**: filtra las tuplas eliminando todos los valores que no cumplan las condiciones dadas.
- **order by**: ordena las tuplas según el criterio dado.
- **return**: construye el resultado de la consulta para una tupla dada, después de haber sido filtrada por la cláusula where y ordenada por la cláusula order by.

En XQuery, el término tupla se refiere a cada uno de los valores que toma una variable.

Sea la siguiente consulta:

```
for $x in doc("libros.xml")/libreria/libro
where $x/precio>30
order by $x/titulo
return $x/titulo
```

Ejemplo01.xq

- o La cláusula for selecciona todos los elementos "libro" bajo "librería" y los coloca en una variable llamada \$x
- o La cláusula where selecciona sólo los elementos libro con un valor en el elemento precio mayor de 30
- o La cláusula order by define el tipo de ordenación. Así, indica que los resultados sean ordenados por el elemento titulo
- o La cláusula return especifica qué es lo que ha de devolverse. En este caso, devuelve los elementos titulo

El resultado de la expresión XQuery anterior es:

```
<titulo leng="en">Learning XML</titulo>
<titulo leng="en">XQuery Start</titulo>
```

4.1 La cláusula "for"

La cláusula "**for**" enlaza una variable con cada item que se devuelve por la expresión "**in**". Esta cláusula produce una iteración. Puede haber múltiples cláusulas "for" en una misma expresión.

- Para conseguir una iteración un determinado número de veces se utiliza la palabra reservada "**to**". Por ejemplo:

```
for $x in (1 to 3)
return <test>{$x}</test>
```

produce el resultado:<test>1</test>
<test>2</test>
<test>3</test>

Ejemplo02.xq

- La palabra reservada **“at”** permite contar la iteración:

```
for $x at $i in doc("libros.xml")/libreria/libro/titulo
return <libro>{$i}. {data($x)}</libro>
```

Ejemplo03.xq

devuelve el resultado:<libro>1. Everyday I
<libro>2. Harry Potter</libro>
<libro>3. XQuery Kick Start</libro>
<libro>4. Learning XML</libro>

- Se permite también más de una expresión en una cláusula for. Para ello, se separan los valores mediante comas:

```
for $x in (10,20), $y in (100,200)
return <test>x={$x} and y={$y}</test>
```

Ejemplo04.xq

devuelve el resultado:<test>x=10 and y=100</test>
<test>x=10 and y=200</test>
<test>x=20 and y=100</test>
<test>x=20 and y=200</test>

4.2 La cláusula **“let”**

La cláusula **“let”** permite hacer asignaciones de variables, permitiendo evitar el tener que repetir la misma expresión varias veces. La cláusula let no provoca una iteración:

```
let $x := (1 to 5)
return <test>{$x}</test>
```

Ejemplo05.xq

produce el resultado:<test>1 2 3

4.3 Las cláusulas **“where”** y **“order by”**

- La cláusula **“where”** se usa para especificar uno o más criterios para el resultado:

```
where $x/precio>30 and $x/precio<100
```

- La cláusula **“order by”** especifica el criterio de ordenación del resultado. Por defecto se ordena por orden ascendente. Si se quiere ordenar por orden descendente se debe incluir la palabra **“descending”** al final. Así, para ordenar ejemplo por categoría y título:

```
for $x in doc("libros.xml")/libreria/libro
order by $x/@categoria, $x/titulo
return $x/titulo
```

Ejemplo06.xq

produciría el resultado:

```
<titulo leng="en">Harry Potter</titulo>
<titulo leng="en">Everyday Italian</titulo>
<titulo leng="en">Learning XML</titulo>
<titulo leng="en">XQuery Kick Start</titulo>
```

4.4 La cláusula “return”

La cláusula “return” indica qué es lo que se quiere devolver:

```
for $x in doc("libros.xml")/libreria/libro
return $x/titulo
```

Ejemplo07.xq

produce como resultado:

```
<titulo leng="en">Everyday Italian</titulo>
<titulo leng="en">Harry Potter</titulo>
<titulo leng="en">XQuery Kick Start</titulo>
<titulo leng="en">Learning XML</titulo>
```

La sentencia return puede devolver más de un valor. En este caso, aparecerán los distintos valores entre paréntesis y separados por comas (,). Por ejemplo:

```
for $x in doc("libros.xml")/libreria/libro
return ($x/titulo, $x/autor, string($x/@categoria))
```

La salida sería:

```
<titulo leng="en">Everyday Italian</titulo>
<autor>Giada De Laurentiis</autor>
COOKING
<titulo leng="en">Harry Potter</titulo>
<autor>J K. Rowling</autor>
CHILDREN
<titulo leng="en">XQuery Kick Start</titulo>
<autor>James McGovern</autor>
<autor>Per Bothner</autor>
<autor>Kurt Cagle</autor>
<autor>James Linn</autor>
<autor>Vaidyanathan Nagarajan</autor>
WEB
<titulo leng="en">Learning XML</titulo>
<autor>Erik T. Ray</autor>
WEB
```

Ejemplo08.xq

También se puede devolver con un return un texto combinando marcas y datos obtenidos en la consulta. Por ejemplo:

```
for $x in doc("libros.xml")/libreria/libro
return <h1>{$x/titulo}</h1>
```

Ejemplo09.xq

Lo que devolvería:

```
<h1><titulo leng="en">Everyday Italian</titulo></h1>
<h1><titulo leng="en">Harry Potter</titulo></h1>
<h1><titulo leng="en">XQuery Kick Start</titulo></h1>
<h1><titulo leng="en">Learning XML</titulo></h1>
```

No es posible devolver un nodo atributo en XQuery. Para devolver el **valor** de un atributo hay que utilizar la función string o data. Por ejemplo:

```
for $x in doc("libros.xml")/libreria/libro
return ($x/titulo, string($x/@categoria))
```

Ejemplo10.xq

Y tendríamos esta conversión:

```
<titulo leng="en">Everyday Italian</titulo>
COOKING
<titulo leng="en">Harry Potter</titulo>
CHILDREN
<titulo leng="en">XQuery Kick Start</titulo>
WEB
<titulo leng="en">Learning XML</titulo>
WEB
```

4.5 Archivos XQuery

Se pueden almacenar consultas XQuery en archivos, que normalmente llevan la extensión “xquery”, o más comúnmente “xq”. Estos archivos suelen comenzar con:

```
xquery version "1.0";
```

En estos archivos se pueden combinar elementos de marcado, datos carácter y expresiones FLOWR (cada una encerrada entre llaves “{ }”). Por ejemplo, el siguiente documento XQuery obtiene la lista de todos los títulos de los libros en forma de tabla html:

```
xquery version "1.0";
<html>
  <body>
    <table border="1">
      {
        for $x in doc("libros.xml")/libreria/libro
        return
          <tr><td>{string($x/titulo)}</td></tr>
      }
    </table>
  </body>
</html>
```

Ejemplo11.xq

Produciría esta salida:

```
<html>
  <body>
    <table border="1"><tr><td>Everyday Italian</td></tr>
      <tr><td>Harry Potter</td></tr>
      <tr><td>XQuery Kick Start</td></tr>
      <tr><td>Learning XML</td></tr>
    </table>
  </body>
</html>
```

5 XQuery: operadores y funciones

XQuery 1.0, XPath 2.0 y XSLT 2.0 comparten el mismo conjunto de operadores y funciones. Se puede ver un listado completo de funciones XQuery en [4]. Hay operadores y funciones de todo tipo: matemáticas, de cadenas, de comparación de fechas y horas, de manipulación de nodos XML, etc. Además de la biblioteca de funciones predefinidas, XQuery permite la creación de funciones definidas por el usuario.

Una llamada a una función puede aparecer en los mismos sitios donde que las expresiones. El prefijo usado por defecto en el espacio de nombres de las funciones es “**fn:**”. Por ejemplo, `fn:string()`. Sin embargo, esto no ocurre en el caso del nombre de las funciones, que no necesitan emplear el prefijo cuando son llamadas.

Veamos algunos ejemplos de funciones predefinidas.

Funciones generales

- **doc(“URI”)**: devuelve el documento completo (el nodo documento)
- **empty(item, item, ...)**: devuelve “true” si el valor del argumento ES una secuencia vacía. De lo contrario devuelve “false”.
- **exists(item1, item2, ...)**: es la función opuesta a “empty”. Devuelve “true” si el valor del argumento NO ES una secuencia vacía. De lo contrario devuelve “false”.
- **distinct-values()**: extrae los valores de una secuencia de nodos y crea una nueva secuencia con valores únicos, eliminando los nodos duplicados.
- **string()**: convierte el argumento (que será un valor atómico o un nodo) a string. Si el argumento es un nodo elemento, devuelve los datos carácter del contenido de elemento y de todos sus descendientes concatenados. Si es un nodo atributo, devuelve el valor del atributo como string. Si es un valor atómico, devuelve ese valor convertido a string.
- **data()**: devuelve el valor atómico del argumento. Si estos son nodos, devuelve sus datos carácter.

Funciones agregadas

`sum`, `avg`, `count`, `max`, `min`, etc., que operan sobre una secuencia de números y devuelven un resultado numérico.

Funciones de cadena

`string-length`, `substring`, `upper-case`, `lower-case()`, `concat()`, etc.

6 Referencias

Recomendaciones W3C

- [1] Página Web sobre XQuery del W3C. <http://www.w3.org/standards/techs/xquery>
- [2] XQuery 1.0: An XML Query Language (Second Edition). W3C Recommendation 14 December 2010 (*Link errors corrected 3 January 2011*). <http://www.w3.org/TR/2010/REC-xquery-20101214/> : Este es el documento principal del proyecto, el cual presenta el lenguaje y da una perspectiva de todos sus aspectos.
- [3] XQuery 1.0 and XPath 2.0 Data Model (XDM) (Second Edition). W3C Recommendation 14 December 2010. <http://www.w3.org/TR/xpath-datamodel/>. Este documento describe una extensión de conjunto de modelo de datos de XML
- [4] XQuery 1.0 and XPath 2.0 Functions and Operators (Second Edition). W3C Recommendation 14 December 2010. <http://www.w3.org/TR/2010/REC-xpath-functions-20101214/>: Este documento detalla las funciones y operadores básicos soportados por XQuery y XPath.
- [5] XML Path Language (XPath) 2.0 (Second Edition). W3C Recommendation 14 December 2010 (*Link errors corrected 3 January 2011*). <http://www.w3.org/TR/xpath20/>

Libros

- [6] XML Primer Plus. Nicholas Chase. Sams Publishing.
- [7] <http://www.agapea.com/libros/XML-practico-Bases-esenciales-conceptos-y-casos-practicos-isbn-2746049589-i.htm>

Tutoriales y otro material

- [8] Tutorial de W3 Schools sobre XQuery. <http://www.w3schools.com/xquery/>
- [9] Lista de funciones XQuery. <http://www.xqueryfunctions.com/xq/>
- [10] XML y Bases de Datos. Presentación de Ana Belén Martínez Prieto, Universidad de Oviedo. <http://www.di.uniovi.es/~labra/cursos/ver06/pres/XMLBD.pdf>
- [11] XQuery. J. J. Gutiérrez, M. J. Escalona, M. Mejías, J. Torres, D.Villadiego. Departamento de Lenguajes y Sistemas Informáticos. Escuela Técnica Superior de Ingeniería Informática, Universidad de Sevilla. <http://www.lsi.us.es/docs/informes/LSI-2005-02.pdf>